

Engañé al gate de IA de tu pipeline: la firma seguía siendo válida y aun así no entró

Seguridad ofensiva sobre un pipeline DevSecOps con agentes de IA

Jean Paul López – Senior Consultant, Red Hat · Lima, Perú

Propuesta para **Black Alpaca 2026** · sesión de 50 min · español · nivel intermedio-avanzado

Laboratorio DevSecOps completo, reproducible y público

github.com/jeanlopez-talks/black-alpaca-peru-redteam-agentes-kubernetes-2026

Pipeline Tekton de 11 etapas · supply chain con firma y procedencia · políticas de admisión · cuatro agentes de IA · portal interno (Backstage). Todo GitOps.

1 · La tesis

DevSecOps lleva años desplazando los controles hacia la izquierda: escaneo de código, SBOM, firma de imágenes, políticas de admisión. Ahora estamos metiendo **agentes de IA** en esa misma cadena – revisan PRs, deciden gates, proponen correcciones – y los integramos en el portal interno de desarrollo para que el equipo trabaje con ellos.

Cada agente es una **identidad nueva** con acceso al código, a las credenciales del pipeline y, en última instancia, a producción. La pregunta ofensiva es directa: ¿qué pasa cuando el atacante le habla al agente que cuida tu código?

Un control que lee texto escrito por el atacante no es un control de seguridad, por bueno que sea el modelo que hay detrás. Lo que aguanta es la plataforma.

La charla lo ataca en vivo sobre un pipeline DevSecOps propio y enseña qué control sí frena a un agente que ya tiene credenciales.

2 · El laboratorio

Cadena DevSecOps completa sobre un clúster **k3s propio**: desde que se abre el PR hasta que la admisión decide si entra. Todo GitOps y reproducible. Sin nube de terceros y sin servicio externo de IA – el modelo corre dentro del clúster, así que ningún fragmento de código sale de la infraestructura.

Pieza	Qué es
Pipeline	Tekton, 11 etapas: clone → pruebas → SAST → build → SBOM → escaneo → firma → procedencia → gate → verificación → despliegue
Supply chain	Registro OCI interno; firma y verificación por digest con cosign; SBOM con Syft; escaneo con Trivy
Admisión	Kyverno: exige firma válida y aprobación humana y que el cambio venga de Argo CD
Agentes	Cuatro, comunicados por A2A: atacante, revisor, remediación y aprobación
Modelo	Qwen3-8B servido con vLLM dentro del clúster : sin salida a internet y sin API key
Portal interno (IDP)	Backstage: donde los agentes publican su análisis y la persona confirma. Es la puerta por la que el equipo trabaja con ellos

3 · Arquitectura

DE QUÉ DEPENDE · servicios del clúster

ninguno sale a internet

Almacén de imágenes
donde vive la imagen, su firma y su certificado

El modelo de IA
corre dentro del clúster, sin salir a internet

Puerta al catálogo
por aquí los agentes leen las políticas de seguridad

El control de admisión
decide qué puede entrar al clúster

EL PIPELINE · aquí corre el código del PR

aquí se ejecuta código que no controlas: sin credenciales del clúster

EL PIPELINE, PASO A PASO · DE IZQUIERDA A DERECHA



QUÉ AGENTE ACTÚA EN CADA ETAPA

estos sí tienen credenciales — y aun así solo proponen, nunca aplican

Backstage · el portal
aquí entra la persona
Lee lo que proponen los agentes y confirma, o no, cada arreglo.

Agente de remediación
actúa en: remediation
Estudia las vulnerabilidades y propone cómo arreglarlas.

Agente de aprobación
observa todas las etapas
¿El revisor acordó o lo engañaron? Da una segunda opinión.

QUIÉN LO GOBIERNA · y quién decide qué entra

solo entra lo que está escrito en Git y aprueba una persona

Argo CD
mantiene el clúster igual a lo que dice Git

el pipeline

los agentes con IA

una ejecución por acto

la aplicación objetivo

las reglas de admisión

El único camino válido: una persona aprueba el cambio en Git, y Argo CD lo aplica. El agente puede falsificar una aprobación, pero no puede hacerse pasar por Argo CD.

4 · Los cuatro actos y su resultado

Acto	Qué hace el atacante	Resultado medido
1	PR malicioso evidente	El revisor lo bloquea . El control funciona cuando el ataque es obvio
2	Inyección indirecta en una dependencia	El revisor es engañado y aprueba
3	Instrucción escondida en el diff	Gana el gate , y la firma de la imagen verifica correctamente : el ataque toca la configuración, no la imagen
4	El mismo ataque contra el revisor endurecido	Bloqueado

El desenlace del acto 3 es el centro de la charla. El revisor aprobó, la firma es válida y el agente **tiene credencial real**. Y aun así el despliegue no entra: la admisión lo rechaza dos veces – primero porque falta la aprobación de una persona; después, cuando el agente se falsifica a sí mismo esa aprobación, porque el cambio no viene de Git.

5 · El hallazgo incómodo

Probé el revisor con el **modelo real** en lugar del motor de reglas:

Con Qwen3-8B local, el revisor **bloqueó las 5 ejecuciones**: ve el comando peligroso escondido tras la nota.

El engaño del acto 3 **solo ocurre con el motor de reglas**, donde obedecer la nota del PR está programado a propósito para que la demo sea reproducible. Lo presento tal cual porque es mejor material que la promesa original:

Un modelo mejor sube el listón. No lo convierte en un control. El atacante itera el encuadre; la admisión no negocia.

6 · Los agentes que sí aportan

El patrón que propongo para usar un modelo dentro de un pipeline sin confiar en él:

El modelo decide · el esquema acota · el código verifica

- El **código** calcula hechos comprobables desde el grafo de dependencias: qué librerías carga de verdad la aplicación, con qué usuario arranca y qué comando ejecuta.
- El **modelo** razona solo sobre esos hechos, con un formato de respuesta cerrado que el servidor de inferencia impone token a token.
- El **código** verifica la respuesta y corrige al modelo si se inventó un paquete, una vulnerabilidad o una acción imposible. Cada corrección queda registrada y se muestra.
- El **parche lo escribe el código**, nunca el modelo.

Resultado medido: la imagen baja de **37 a 13 vulnerabilidades graves**, con la aplicación funcionando – configuración válida y respuesta HTTP 200 – verificado reescaneando.

Y la frontera que importa, que es donde entra el **portal interno de desarrollo**: el agente publica su análisis en Backstage con todo lo que usó para decidir, una persona lo revisa y **confirma**, y solo entonces el cambio se escribe en Git para que lo aplique Argo CD. **El agente propone; nunca toca el clúster.**

7 · Esquema de la sesión (50 min)

Min	Bloque
0-5	Los agentes ya están en tu pipeline: cada uno es una identidad con acceso a producción
5-12	El escenario: el pipeline de 11 etapas y los agentes que actúan en él
12-18	Acto 1 – el revisor bloquea el ataque obvio
18-26	Actos 2 y 3 – la instrucción escondida gana el gate, y la firma verifica igual
26-32	El hallazgo: con el modelo real, 5 de 5 bloqueados. Un modelo mejor no es un control
32-40	Lo que sí frena: la admisión rechaza al agente dos veces
40-47	Los agentes que aportan: 37 → 13 vulnerabilidades, y la persona confirma
47-50	Checklist para gobernar los agentes de tu pipeline. Preguntas

8 · Qué se lleva quien asiste

- Cómo se envenena un gate de IA que lee texto escrito por el atacante.
- Por qué firmar y escanear son **necesarios y no suficientes**: la cadena de supply chain puede estar intacta mientras el ataque sigue vivo.
- Dónde poner la frontera cuando se integran agentes en el pipeline y en el **portal interno de desarrollo**: qué pueden proponer y qué solo puede confirmar una persona.
- Un pipeline DevSecOps **reproducible en GitOps** para probarlo en su propia infraestructura.

9 · Honestidad sobre el alcance

- Los resultados de este laboratorio son **propios**, medidos en mi clúster.
- La metodología de aislamiento parte del estudio de **Roy Belio (Red Hat)**, citado como fuente primaria; sus números no se presentan como míos.
- El ataque de los 3 000 millones de tokens (mayo 2026) se narra como **caso de estudio citado**, nunca replicado.
- La demo corre en vivo, con **grabación de respaldo** por si falla la red o la GPU.

Todo el material de esta propuesta es público

github.com/jeanlopez-talks/black-alpaca-peru-redteam-agentes-kubernetes-2026

El pipeline DevSecOps y las políticas de admisión como manifiestos GitOps · el código de los cuatro agentes · el plugin del portal interno · el diagrama interactivo · las diapositivas · las evidencias de cada medición.

El repositorio reúne las dos propuestas de esta edición; el material de esta sesión está en [session-2-ai-red-vs-blue-devsecops/](#).